Project 1 — Transformations, Projection & Curves from Paintings

Released	Thursday 11 October 2026 (with starter code)
Due	Sunday 8 November 2026, 23:59 (Istanbul time)
Work	Individually. Code on GitHub, report as PDF.
Weight	Graded out of 100. Projects together are 40 % of the course grade.
Support	Mon 5 Oct walkthrough and Thu 8 Oct clinic (TA); labs on 7, 14, 21 Oct and 4 Nov

1 · Overview

You will build one interactive WebGPU application that turns a painting from the Sakıp Sabancı Museum collection into a small 3D scene. The painting itself hangs in the scene as a textured quad; in front of it you place objects, move a camera around them, and trace curves over the painted shapes.

Everything you need for this project is a consequence of one chain:

$$\mathbf{p_clip} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{p_object}$$

Part A builds M, Part B composes many M's in a tree, Part C builds V and P, Part D adds geometry that is generated rather than modelled, and Part E puts it together. The maths is the assignment: you write the matrix and curve code yourself.

Interactive specification: open `Project1/index.html` from the course folder. Every part below has a live demo there showing exactly what the result should do.

2 · Learning objectives

- Implement 4×4 affine transformations and explain why their order matters.
- Build and traverse a scene graph, and derive a world matrix from a chain of local ones.
- Derive a view matrix with cross products and a projection matrix for WebGPU's clip space.
- Evaluate Bézier and spline curves, and reason about continuity between segments.
- Write a complete WebGPU render loop: buffers, uniforms, pipelines, draw calls.
- Report technical work: derivations, evidence, measurements.

3 · Choosing your painting

Pick one painting from the SSM collection. Two kinds work well:

- **Strong perspective cues** (Hoca Ali Rıza — Moda İskelesi; Feyhaman Duran — Kabataş Rıhtımı; Şevket Dağ — II. Selim Türbesi Kapısı): good for camera work — you can match your virtual camera to the painted one.
- **Strong organic contours** (Şeker Ahmed Paşa — Kavunlu Natürmort; Süleyman Seyyid — Elmalı Natürmort): good for curve tracing.

State your choice, with the artist and title, on the first page of your report. Two students may choose the same painting; identical scenes are not acceptable.

4 · Environment and rules

- **WebGPU and WGSL only.** No Three.js, no Babylon, no gl-matrix, no other maths or scene-graph library. You may use a small UI library (e.g. lil-gui) for sliders.
- JavaScript or TypeScript, served by Vite (the starter repository is set up).

- Matrices are column-major and uploaded to uniform buffers exactly as WebGPU expects them.
- Your code must run in a current Chrome, Edge, Safari or Firefox without a build step beyond `npm install && npm run dev`.
- Work individually. Discussing ideas is fine; sharing code is not.

What the starter code gives you

- A working WebGPU context: device request, canvas configuration, resize handling, render loop.
- Your painting loaded as a texture and drawn on a quad, with correct sRGB handling.
- An empty maths module with function signatures and unit tests you have to make pass.
- A minimal UI panel and a frame-time counter.

Part A · Transformations — 20 points

Write your own matrix library and demonstrate composition order.

- Implement: identity, multiply, translate, rotate about x/y/z, scale, transpose, and the inverse of a rigid transform (rotation + translation only).
- Place at least three objects in front of the painting using these matrices.
- One object must be controlled from the UI: translation, rotation angle and scale.
- Provide a switch between $M = T \cdot R \cdot S$ and $M = S \cdot R \cdot T$, and display both matrices numerically on screen.

Evidence: screenshots of both orders with the matrices visible, and a paragraph in the report explaining the difference and stating which convention the rest of your code uses.

Part B · Scene graph — 15 points

Objects that belong together should move together.

- A node type holding a local transform, a parent, children, and a world matrix computed by traversal.
- At least three levels of hierarchy and at least five nodes.
- At least one animated node, so the hierarchy is visible in motion (for example a rotating arm carrying an object).
- A debug mode drawing each node's local axes.

Evidence: a diagram of your tree in the report, plus the world matrix of one leaf node written out and verified by hand.

Part C · Camera and projection — 20 points

- Implement lookAt(eye, target, up) using two cross products; handle the degenerate case where up is parallel to the view direction.
- Implement perspective(fovY, aspect, near, far) mapping depth to $z \in [0, 1]$ — WebGPU's convention, not OpenGL's — and orthographic(...) with the same near/far behaviour.
- Orbit the camera with the mouse; show eye, target, fov, near and far on screen.
- Add a second, fixed "inspection" camera and draw the first camera's frustum as seen from it.
- Switch between perspective and orthographic projection at run time.

Evidence: the derivation of the view matrix in the report; screenshots of the same scene under both projections; the frustum view. Discuss the near/far values you chose and their effect on depth precision.

Part D · Curves over the painting — 25 points

- Cubic Bézier evaluation, and a visualisation of the de Casteljau construction for an adjustable parameter t.
- A piecewise curve of at least three segments with C^1 continuity enforced at the joints.
- One of: B-spline basis evaluation, or a Catmull–Rom spline passing through the control points.

- Control points placed and dragged with the mouse, in the plane of the painting, tracing a shape that is actually in the picture.
- An object animated along the curve, oriented by the curve tangent.
- Export the curve as JSON with control points in normalised coordinates.

Evidence: the Bézier basis functions in the report; a proof that the curve starts at P_0 and ends at P_3; the C^1 condition at a joint; a screenshot of your traced shape over the painting; the exported JSON as an appendix.

Part E · Integration, interface and performance — 10 points

- One application containing parts A–D — not four separate pages.
- Controls to switch views, reset the scene and toggle the debug drawings.
- A frame-time readout and the number of vertices submitted per frame.
- Modular code: separate maths, scene, rendering and UI; short comments where the maths is not obvious.
- A README explaining how to run it and what each control does.

Report

10 points. Three to five pages, PDF. Include: your painting and why you chose it; the derivations named above; screenshots of each part; a short section on what went wrong and how you found it; measured frame times on your machine (state the GPU); and the

Bonus — up to +10

- Fit a curve to a shape you trace by hand (least-squares fitting of a cubic Bézier).
- Build a surface of revolution or an extrusion by sweeping your curve, and render it as a mesh.
- Match your camera to the painting's own perspective and argue the choice in the report.
- Quaternion-based orbit camera with smooth interpolation between viewpoints.
- Evaluate the curve in a compute shader and render it as a tessellated ribbon.

Deliverables and submission

- A GitHub repository (link submitted on SUCourse) containing the application, a README and the JSON export.
- A tag or release named project1-final created before the deadline; the tag timestamp is what is graded.
- report.pdf, in the repository and uploaded separately to SUCourse.
- Optional: a demo video of at most three minutes.

Late submissions lose 10 points per started day, up to three days; after that the project is graded as zero. [Confirm with the instructor.]

Grading rubric

Criterion	What is assessed	Points
A · Transformations	Correct matrix maths; both composition orders demonstrated and explained	20
B · Scene graph	Hierarchy of at least three levels; correct world matrices; animated node	15
C · Camera & projection	lookAt derivation; WebGPU-correct projection; frustum visualisation; both projections	20
D · Curves	Bézier and de Casteljau; C^1 piecewise curve; second curve family; interaction; animation along the curve	25

Criterion	What is assessed	Points
E · Integration	One coherent application; controls; performance readout; code structure	10
Report	Derivations, evidence, measurements, honest discussion	10
Bonus	Any of the extensions above, done well	+10

Work that does not run is graded on what can be read in the source, which is always worse than a smaller feature set that runs.